Trabajo práctico 4

Ejercicio 1

① Tome una de las variables de la línea 3 del siguiente código e indique y defina cuales son sus atributos:

```
1. Procedure Practica4();
2. var
3. a,i:integer
4. p:puntero
5. Begin
6.   a:=0;
7.   new(p);
8.   p:= ^i
9.   for i:=1 to 9 do
10.    a:=a+i;
11.  end;
12.  ...
13.  p:= ^a;
14.  ...
15.  dispose(p);
16.end;
```

① Compare los atributos de la variable del punto a) con los atributos de la variable de la línea 4. Que dato contiene esta variable?,

Inciso 1.1

Identificador	Tipo	r-valor	Alcance	T.V
a	integer	0 → 45	3-16	1-16

Inciso 1.2

Identificador	Tipo	r-valor	Alcance	T.V
p	puntero	nil → direccion de i → direccion de a	4-16	1-16

Ejercicio 2

① Indique cuales son las diferentes formas de inicializar una variable en el momento de la declaración de la misma.

② Analice en los lenguajes: Java, C, Phyton y Ruby las diferentes formas de inicialización de variables que poseen. Realice un cuadro comparativo de esta característica.

Inciso 2.1

La inicialización es el proceso de establecer una ligadura entre una variable y su r-valor (valor codificado) en el momento en que se introduce la entidad en el programa. Las fuentes identifican tres estrategias principales:

- **Inicialización en la declaración:** El programador especifica el valor inicial explícitamente al momento de declarar la variable.
- **Inicialización por defecto:** El lenguaje asigna automáticamente un valor predefinido según el tipo. Por ejemplo, los **enteros se inicializan en 0**, los caracteres en blanco y ciertas funciones en `VOID`.
- **Estrategia de "Ignorar el problema":** El lenguaje no realiza ninguna acción y la variable toma como valor inicial lo que sea que haya quedado en la celda de memoria asignada (cadena de bits previa). Esto puede generar errores semánticos y requiere chequeos adicionales por parte del programador.

Inciso 2.2

Análisis por lenguaje: Java, C, Python y Ruby

Basándonos en la clasificación de los lenguajes según su modelo de ejecución y tipado presentes en las fuentes, se analiza su comportamiento:

- **C:** Es un lenguaje de **compilación pura** y **alcance estático**. Soporta la inicialización explícita en la declaración. Para variables locales (automáticas), suele utilizar la estrategia de "**ignorar el problema**", mientras que para variables estáticas puede aplicar inicialización por defecto.
- **Java:** Tiene un modelo de ejecución **basado en pilas**. Permite la inicialización en la declaración y aplica **inicialización por defecto** para los atributos de clase (campos), aunque el análisis semántico estático exige que las variables locales sean inicializadas antes de su uso para evitar errores.
- **Python y Ruby:** Son lenguajes **interpretados** con un modelo de ejecución **dinámico** (espacio de memoria en la *heap*). En estos lenguajes, la "declaración" y la "inicialización" ocurren simultáneamente en el momento de la primera asignación de valor, debido a su **tipado dinámico**, donde el tipo se infiere del r-valor en tiempo de ejecución.

Ejercicio 3

Explique los siguientes conceptos asociados al atributo l-valor de una:
a. Variable estática.
b. Variable automática o semiestática.
c. Variable dinámica.
d. Variable semidinámica.
De al menos un ejemplo de cada uno.
Investigue sobre que tipos de variables respecto de su l-valor hay en los lenguajes C y Ada.

Respuesta

a. Variable Estática

Es aquella cuyo l-valor se liga **antes de la ejecución** (durante la compilación o la carga del programa en memoria) y esta ligadura perdura hasta que el programa finaliza. Se alojan en una zona de datos fija.

- **Ejemplo:** En el lenguaje C, una variable declarada con el prefijo `static`, como `static int aux;`, mantiene su dirección de memoria y su r-valor durante toda la vida del programa.

b. Variable Automática

Su l-valor se liga de forma **automática** en el momento en que aparece su declaración durante la secuencia de ejecución. Generalmente se gestionan en una **pila** (stack), alocándose al entrar en su alcance y desalocándose al salir del mismo.

- **Ejemplo:** Las variables locales comunes en lenguajes como C o Java, por ejemplo `int x, y;` declaradas dentro de una función, son automáticas.

c. Variable Dinámica

Son variables cuyo l-valor es solicitado **explícitamente por el programador** mediante una sentencia de creación o constructor en tiempo de ejecución. Se alocan en una zona de memoria llamada **heap** y se acceden usualmente a través de punteros.

- **Ejemplo:** El uso de punteros para crear objetos en la heap, como en la instrucción `new(p)` de Pascal o Ada.

d. Variable Semidinámica

Es una variante donde la ligadura al l-valor ocurre en tiempo de ejecución (similar a las automáticas), pero ciertos atributos como el tamaño de la estructura (por ejemplo, un arreglo) se definen en el momento de la alocación y no cambian después.

- **Ejemplo:** Los **arreglos semidinámicos en Ada**, donde el tamaño se determina al entrar al bloque donde se declaran.

Tipos de variables respecto de su l-valor hay en los lenguajes C y Ada

- **Lenguaje C:**
  - **Estáticas:** Utiliza la palabra clave `static` para variables cuyo l-valor es persistente durante toda la ejecución.
  - **Automáticas:** Es el tipo por defecto para las variables locales declaradas dentro de funciones.
  - **Dinámicas:** Se gestionan mediante punteros y funciones de alocación de memoria en el heap.
  - En C, el tiempo de ejecución permite expresiones que involucran variables donde la ligadura ocurre al crearse la variable.

- **Lenguaje Ada:**
  - **Semidinámicas:** Es una característica distintiva de este lenguaje, específicamente en la gestión de arreglos cuyo tamaño se define en tiempo de ejecución.
  - **Estáticas y Automáticas:** Al ser un lenguaje de compilación pura con un modelo basado en pilas, utiliza variables estáticas para datos globales y automáticas para locales.
  - **Dinámicas:** Al igual que C, permite la creación de variables en la heap mediante punteros (access types).

Ambos lenguajes realizan la **traducción antes de la ejecución** (compilación pura), lo que les permite gestionar de manera eficiente estas ligaduras de memoria antes de que el hardware ejecute el código directamente.

Ejercicio 4

① ¿A qué se denomina variable local y a qué se denomina variable global?
② ¿Una variable local puede ser estática respecto de su l-valor? En caso afirmativo dé un ejemplo
③ Una variable global ¿siempre es estática? Justifique la respuesta.
④ Indique qué diferencia hay entre una variable estática respecto de su l-valor y una constante

Inciso 4.1

- **Variable Local:** Es aquella cuya referencia se crea **dentro de una unidad** (subprograma o función). Su nombre solo es conocido y visible dentro de ese bloque, y su ligadura suele destruirse al finalizar la ejecución de dicha unidad.
- **Variable Global:** Es aquella cuya referencia se crea en el **Programa Principal**. Es visible y puede ser referenciada desde cualquier parte del programa.

Inciso 4.2

Sí, una variable puede tener un alcance local (su nombre solo se conoce en una función) pero poseer un **l-valor estático** (su dirección de memoria persiste durante toda la ejecución del programa).

- **Ejemplo:** En el lenguaje **C**, una variable declarada con la palabra reservada `static` dentro de una función:
- Aquí, el nombre de dicha variable solo existe dentro de `funcion`, pero el espacio de memoria (l-valor) se reserva al inicio y no se libera hasta que el programa termina.

Inciso 4.3

Desde la perspectiva del **l-valor**, la respuesta es **sí**. Una variable se considera estática respecto a su l-valor cuando la ligadura a su dirección de memoria se establece antes de la ejecución (o al cargar el programa) y **perdura hasta el fin de la misma**. Dado que las variables globales deben estar disponibles para todo el programa desde su inicio hasta su

cierre, su tiempo de vida (extensión) coincide con la duración total del programa, cumpliendo con la definición de variable estática.

Inciso 4.4

La diferencia principal radica en la estabilidad de su **r-valor** (el valor almacenado):

- **Variable Estática:** Aunque su l-valor (ubicación) es fijo y no cambia, su **r-valor es dinámico**; es decir, el valor contenido en esa dirección de memoria puede ser modificado mediante sentencias de asignación durante la ejecución.
  - Una **variable estática** posee un **l-valor estático**
- **Constante:** Su r-valor se liga en tiempo de compilación o ejecución, pero **no puede ser modificado** una vez establecido. Además, el compilador a menudo optimiza las constantes reemplazando el nombre directamente por el valor, mientras que las variables estáticas siempre mantienen una celda de memoria accesible.
  - se caracterizan por una **ligadura estática** o fija que se establece antes o al inicio de la ejecución y no puede ser modificada

Ejercicio 5

① En Ada hay dos tipos de constantes, las numéricas y las comunes. Indique a que se debe dicha clasificación.

② En base a lo respondido en el punto a), determine el momento de ligadura de las constantes del siguiente código:
H: constant Float:= 3,5;
I: constant:= 2;
K: constant float:= H*I;

Inciso 5.1

La clasificación de las constantes en Ada entre numéricas y comunes se debe principalmente al momento de ligadura (binding time) de su r-valor y a la naturaleza de la expresión que las inicializa.
A continuación se detalla a qué responde cada categoría según las fuentes:

- Constantes Numéricas: Su ligadura es estática y se establece durante el tiempo de compilación. En este nivel, el compilador reemplaza directamente el nombre de la constante por su valor literal en cada punto donde se referencia, lo que resulta en un programa más eficiente al permitir optimizaciones directas.
- Constantes Comunes: Su ligadura es dinámica y ocurre en el tiempo de ejecución (runtime), en el momento exacto en que se crea la variable en la memoria. La razón de esta clasificación es que Ada permite que estas constantes sean inicializadas mediante expresiones que involucran variables.

Inciso 5.2

- **H (Compilación):** Ocurre en **tiempo de ejecución** (runtime). Incluso si le asignas un valor literal directo, la semántica de Ada establece que esta asociación se realiza en el momento

en que **se crea la variable** (o constante) durante la ejecución del programa
- **I (Compilación):** Su ligadura ocurre en **compilación**. Este identificador funciona como una "constante numérica" (o nombre de número).
- **K (Ejecución):** El momento de ligadura es **dinámico** (**en ejecución**). Según las fuentes, Ada permite que las constantes se definan mediante **expresiones** (en este caso el producto `H*I`). En estos casos, la ligadura del r-valor ocurre recién en tiempo de ejecución, cuando se crea la entidad en memoria, para brindar una mayor flexibilidad al programador.

Ejercicio 6

Sea el siguiente archivo con funciones de C:

```
Archivo.c
{ int x=1; (1)

     int func1();{
          int i;
          for (i:=0; i < 4; i++) x=x+1;
     }

     int func2();{
          int i, j;
          /*sentencias que contienen declaraciones y
          sentencias que no contienen declaraciones*/
          ......
          for (i:=0; i < 3; i++) j=func1 + 1;
     }
}
```

Analice si llegaría a tener el mismo comportamiento en cuanto a alocación de memoria, sacar la
declaración (1) y colocar dentro de func1() la declaración static int x =1;

Al realizar el cambio de una **variable global** a una **variable local estática** en C, el comportamiento respecto a la **alocación de memoria** (**l-valor**) es efectivamente el mismo, pero se producen cambios críticos en otros atributos.

Comportamiento de la Alocación (**L-valor**)

En ambos casos, el comprtamiento es **idéntico** en cuanto al momento de ligadura:

- **Ligadura Estática:** Tanto la variable global `x` como la local `static int x` tienen una ligadura de l-valor (dirección de memoria) **estática**. Esto significa que la memoria se reserva **antes de la ejecución** (en tiempo de compilación o carga).
- **Tiempo de Vida** (**Extensión**)**:** Ambas variables tienen el mismo tiempo de vida. La memoria permanece alocada desde que inicia el programa hasta que este **finaliza su**

ejecución

Diferencia en el Alcance (Visibilidad)

Es donde el comportamiento cambia radicalmente:

- **Variable Global (Original):** El nombre `x` tiene un **alcance global**. Es visible y puede ser referenciada por cualquier unidad dentro del archivo, incluyendo `func1()` y `func2()`.
- **Variable Local Estática (Modificada):** Al colocarla dentro de `func1()`, su alcance se vuelve **local**. El identificador `x` solo es conocido y visible dentro de las instrucciones de `func1()`. Si `func2()` intentara usar el nombre `x`, el compilador reportaría un error porque el nombre sería invisible fuera de `func1()`.

Resumen: Si solo analizamos la **alocación de memoria**, la respuesta es **sí, tienen el mismo comportamiento**, ya que ambas son variables estáticas respecto a su l-valor. La diferencia es puramente de **alcance**, limitando quién puede "ver" y modificar esa celda de memoria a través del nombre

Ejercicio 7

Sea el siguiente segmento de código escrito en Java, indique para los identificadores si son globales o locales.

```
Clase Persona {

public long id

public string nombreApellido

public Domicilio domicilio

private string dni;

public string fechaNac;

public static int cantTotalPersonas;

//Se tienen los getter y setter de cada una de las variables

//Este método calcula la edad de la persona a partir de la fecha de
nacimiento
     public int getEdad(){
          public int edad=0;
          public string fN =
          this.getFechaNac();
          return edad;
     }

}
```

```
Clase Domicilio {

public long id;

public static int nro

public string calle

public Localidad loc;

//Se tienen los getter y setter de cada una de las variables
}
```

Clase persona

Atributo	Alcance	Justificación
id	global	Atributo de clase.
nombreApellido	global	Atributo de clase.
domicilio	global	Atributo de clase.
dni	global	Aunque sea `private`, está definida a nivel de clase, no dentro de un método.
fechaNac	global	Atributo de clase.
cantTotalPersonas	global	Atributo de clase(estático)
edad	local	Definida dentro del método `getEdad()`
fN	local	Definida dentro del método `getEdad()`

Clase domicilio

Atributo	Alcance	Justificación
id	Global	Atributo de clase.
nro	Global	Atributo de clase(estático).
calle	Global	Atributo de clase.
loc	Global	Atributo de clase.

En java:

- **Globales** → todo lo definido en la clase (atributos, sean `public`, `private` o `static`)

- **Locales** → variables creadas dentro de métodos (`edad`, `fN`)

Ejercicio 8

Sea el siguiente ejercicio escrito en Pascal

```
1- Program Uno;
2- type tpuntero= ^integer;
3- var mipuntero: tpuntero;
4- var i:integer;
5- var h:integer;
6- Begin
7-    i:=3;
8-    mipuntero:=nil;
9-    new(mipuntero);
10-   mipunterno^:=i;
11-   h:= mipuntero^+i;
12-   dispose(mipuntero);
13-   write(h);
14-   i:= h- mipuntero;
15- End.
```

① Indique el rango de instrucciones que representa el tiempo de vida de las variables i, h y mipuntero.

② Indique el rango de instrucciones que representa el alcance de las variables i, h y mipuntero.

③ Indique si el programa anterior presenta un error al intentar escribir el valor de h. Justifique

④ Indique si el programa anterior presenta un error al intentar asignar a i la resta de h con mipuntero. Justifique

⑤ Determine si existe otra entidad que necesite ligar los atributos de alcance y tiempo de vida para justificar las respuestas anteriores. En ese caso indique cuál es la entidad y especifique su tiempo de vida y alcance.

⑥ Especifique el tipo de variable de acuerdo a la ligadura con el l-valor de las variables que encontró en el ejercicio.

Inciso 8.1

Variable	T.V
i	1-15
h	1-15
mipuntero	1-15

Inciso 8.2

Variable	Alcance
i	4-15
h	5-15
mipuntero	3-15

Inciso 8.3

No hay error, debido a que previamente se reserva y asigna valor a mipuntero. La operación es válida porque se accede al r-valor del puntero, que es un entero. Al sumarle i, que es un entero, la operación tiene un resultado exitoso

Inciso 8.4

Sí, posee un error. Este es de tipo semántico.
Se intenta realizar una operación aritmética entre tipos incompatibles. `h` es de tipo `integer` y `mipuntero` es de tipo `tpuntero`. Restar una dirección de memoria (r-valor del puntero) a un entero viola la compatibilidad de tipos del lenguaje Pascal. Además, tras el `dispose`, el r-valor de `mipuntero` es indefinido o nulo.

Inciso 8.5

Existe una entidad adicional que no tiene nombre: la **variable anónima** creada en el *heap*.

- **Entidad:** Variable dinámica (celda de memoria para el entero apuntado).
- **Alcance: Ninguno**. Al ser anónima, no tiene un nombre referenciable directamente en el código; solo se accede a través del r-valor de `mipuntero`.
- **Tiempo de Vida: 9-12**. Comienza con la instrucción `new(mipuntero)` (alocación explícita) y termina con `dispose(mipuntero)` (desalocación explícita)

Inciso 8.6

De acuerdo a cuándo y cómo se liga la dirección de memoria, las variables se clasifican así:

- **i, h, mipuntero:** Son variables **automáticas** (o estáticas de programa principal). Su l-valor se liga al comenzar la unidad (el programa) y se libera automáticamente al terminar.
- **mipuntero^ (Variable Anónima):** Es una variable **dinámica**. Su l-valor se liga por un requerimiento explícito del programador mediante un constructor (`new`) y su tiempo de vida es controlado manualmente.

Ejercicio 9

Elija un lenguaje y escriba un ejemplo:

① En el cual el tiempo de vida de un identificador sea mayor que su alcance
② En el cual el tiempo de vida de un identificador sea menor que su alcance
③ En el cual el tiempo de vida de un identificador sea igual que su alcance

A continuación, presento los ejemplos solicitados utilizando el lenguaje de programación**C**

Inciso 9.1

Esto ocurre con las **variables locales estáticas**. La memoria (l-valor) se reserva al iniciar el programa y no se libera hasta que este termina, pero el nombre solo puede usarse dentro de la función donde se declaró.

```
void contador() {
    static int x = 0; // T.V: Todo el programa | Alcance: Solo dentro de contador()
    x++;
}
```

Inciso 9.2

Este es el caso de las **variables dinámicas** (**anónimas**) accedidas por punteros cuando se libera la memoria antes de que el puntero salga de su alcance.

```
void ejemplo() {
    int *p = malloc(sizeof(int)); // Se crea una variable anónima
    free(p); // El tiempo de vida de la variable anónima termina aquí
    // El nombre 'p' sigue estando en alcance hasta el fin de la función,
    // pero apunta a una memoria que ya no existe (dangling reference).
}
```

- El **alcance** del identificador `p` llega hasta el final de la función, pero el **tiempo de vida** del área de memoria a la que hace referencia (la variable anónima) terminó prematuramente con la instrucción `free`.

Inciso 9.3

Este es el comportamiento estándar de las **variables automáticas** o locales comunes.

```
void suma() {
    int a = 5; // Se aloca al entrar y se libera al salir de la función
}
```

- **Justificación:** El **l-valor** se liga automáticamente cuando aparece la declaración en la secuencia de ejecución y se destruye (desaloca) en el momento en que el alcance termina (al salir del bloque o unidad).

Ejercicio 10

Si tengo la siguiente declaración al comienzo de un procedimiento:
int c; en C
var c:integer; en Pascal

c: integer; en ADA
Y ese procedimiento NO contiene definiciones de procedimientos internos. ¿Puedo asegurar que el alcance y el tiempo de vida de la variable “c” es siempre todo el procedimiento en donde se encuentra definida?. Analícelo y justifique la respuesta, para todos los casos.

Respuesta

Tiempo de Vida (L-valor)

En este atributo, la respuesta es **SÍ** para los tres lenguajes.

- **Justificación:** Las variables declaradas al comienzo de un procedimiento en C, Pascal y Ada se clasifican como **variables automáticas**. Estos lenguajes utilizan un **modelo de ejecución basado en pilas**.
- **Alocación y Desalocación:** La ligadura del **l-valor** (dirección de memoria) se establece automáticamente en el momento en que se invoca el procedimiento y se reserva el espacio en el registro de activación. Esta memoria permanece ligada hasta que la ejecución de la unidad finaliza y se libera la pila.
- Por lo tanto, mientras el procedimiento esté en ejecución (activo), la variable "c" tiene una celda de memoria asociada, garantizando que su **tiempo de vida** sea todo el procedimiento.

Alcance (Nombre)

En este atributo, la respuesta es **NO**; no se puede asegurar que el nombre sea visible en todas las instrucciones del procedimiento.

- **Justificación (Fenómeno de Hiding):** El alcance es el rango de instrucciones donde el nombre es conocido y visible. Los tres lenguajes mencionados utilizan **alcance estático o léxico**, donde la visibilidad depende de la estructura del código.
- **Ocultamiento por Ámbitos Anidados:** Aunque el procedimiento no tenga procedimientos internos definidos, estos lenguajes permiten la creación de **bloques internos** (por ejemplo, mediante llaves `{}` en C o bloques `declare` en Ada).
- **Regla de Visibilidad:** Un ámbito más interno tiene acceso a los nombres de un ámbito más externo, **a menos que** el ámbito interno contenga una definición con el mismo nombre. Si dentro de un bloque interno se declara otra variable `c`, esta **oculta** a la definición original.
- **Consecuencia:** Durante las instrucciones de ese bloque interno, el nombre "c" de la declaración original no es visible ni puede ser referenciado, por lo que su **alcance se interrumpe**, aunque su **tiempo de vida continúe** (ya que la variable original sigue ocupando su lugar en memoria).

Ejercicio 11

1. Responda Verdadero o Falso para cada opción. El tipo de dato de una variable es?
    1. Un string de caracteres que se usa para referenciar a la variable y operaciones que se pueden realizar sobre ella.

    ② Conjunto de valores que puede tomar y un rango de instrucciones en el que se conoce el nombre.
    ③ Conjunto de valores que puede tomar y lugar de memoria asociado con la variable.
    ④ Conjunto de valores que puede tomar y conjunto de operaciones que se pueden realizar sobre esos valores.

② Escriba la definición correcta de tipo de dato de una variable.

Inciso 11.1

① Falso
② Falso
③ Falso
④ V

Inciso 11.2

El **tipo de una variable** se define como la especificación del **conjunto de valores posibles** que se pueden asociar a la variable y del **conjunto de operaciones permitidas** sobre ellos (como crear, acceder o modificar)

Ejercicio 12

Sea el siguiente programa en ADA, completar el cuadro siguiente indicando para cada variable de que tipo es en cuanto al momento de ligadura de su l-valor, su r-valor al momento de alocación en memoria y para todos los identificadores cuál es su alcance y cual es su el tiempo de vida.
Indicar para cada variable su r-valor al momento de alocación en memoria

```
1.    with text_io; use text_io;
2.    Procedure Main is;
3.    type vector is array(integer range <>);
4.    a, n, p:integer;
5.    v1:vector(1..100);
6.    c1: constant integer:=10;
7.    Procedure Uno is;
-
1. type puntero is access integer;
2. v2:vector(0..n);
3. c1, c2: character;
4. p,q: puntero;
5. begin
     7.5.1. n:=4;
     7.5.2. v2(n):= v2(1) + v1(5);
     7.5.3. p:= new puntero;
     7.5.4. q:= p;
     7.5.5. .......
     7.5.6. free p;
```

```
    7.5.7. ......
    7.5.8. free q;
    7.5.9. ......
  7.6. end;
---
8. begin
9. n:=5;
10. .....
11. Uno;
12. a:= n + 2;
13. .....
14. end
```

Aclaración:
Ident.= Identificador / Tipo es el tipo de la variable respecto del l-value
T.V. = Tiempo de Vida / r-valor debe ser tomado al momento de la alocación en memoria.
El alcance de los identificadores debe indicarse desde la línea siguiente a su declaración.

Identificador	**Tipo (L-value)**	**r-valor (alocación)**	**Alcance**	**T.V.**
a (linea 4)	automática	basura	5-14	1-14
n(linea 4)	automática	basura	5-14	1-14
p(linea 4)	automática	basura	5-14	1-14
v1(linea 5)	automática	basura	6-14	1-14
c1(linea 6)	estática	10	7-14	1-14
v2(linea 7.2)	Semidinámica	basura	7.3-7.6	7-7.6
c1(linea 7.3)	automática	basura	7.4-7.6	7-7.6
c2(linea 7.3)	automática	basura	7.4-7.6	7-7.6
p (linea 7.4)	Automática	null	7.5-7.6	7-7.6
q(linea 7.4)	Automática	null	7.5-7.6	7-7.6
Anónima	**Dinámica**	Basura	No tiene	7.5.3 - 7.5.6

Ejercicio 13

El nombre de una variable puede condicionar:

① Su tiempo de vida.
② Su alcance.
③ Su r-valor.
④ Su tipo.
Justifique la respuesta

Respuesta:
El **nombre** de una variable puede condicionar principalmente su **alcance** y su **tipo**, dependiendo de las reglas del lenguaje específico.

Su Alcance (Opción 2)

El nombre puede condicionar el alcance porque, según las fuentes, los lenguajes adoptan diferentes reglas para **ligar el nombre de una variable a su alcance**.

- **Justificación:** El alcance se define como el rango de instrucciones en el que se conoce el nombre. En ciertos lenguajes, la estructura del nombre (como el uso de prefijos o convenciones de mayúsculas/minúsculas) determina si la variable es visible de forma **global o local**. Por ejemplo, en algunos lenguajes interpretados como Ruby (mencionados en el material), el uso de caracteres especiales en el nombre define su visibilidad en diferentes ámbitos.

Su Tipo (Opción 4)

El nombre puede condicionar el tipo de la variable a través de lo que se conoce como **ligadura implícita**.

- **Justificación:** En la ligadura de tipo **implícita**, el tipo de la variable no se declara explícitamente, sino que **se deduce por** "**reglas propias del lenguaje**" basándose en el contexto o en el propio nombre de la variable. Históricamente, lenguajes como FORTRAN (citado como lenguaje de alto nivel en las fuentes) utilizaban la primera letra del nombre para determinar si la variable era de tipo entero o real.

Otras opciones:

- **Tiempo de Vida** (**Opción 1**)**:** Este atributo está ligado al **l-valor** (lugar de memoria) y se define por el momento en que se reserva y libera la memoria (alocación/desalocación), no por el nombre.
- **R-valor** (**Opción 3**)**:** El r-valor es el **valor codificado almacenado** en la ubicación de la variable. El nombre solo sirve para referenciar ese valor, pero no condiciona cuál es el contenido de la celda de memoria.

Ejercicio 14

Sean los siguientes archivos en C, los cuales se compilan juntos
Indicar para cada variable de que tipo es en cuanto al momento de ligadura de su l-valor.
Indicar para cada identificador cuál es su alcance y cual es su el tiempo de vida.
Indicar para cada variable su r-valor al momento de alocación en memoria

```
ARCHIVO1.C
1.int v1;
2.int *a;
3.Int fun2 ()
4.{ int v1, y;
```

```
5.  for(y=0; y<8; y++)
6.  { extern int v2;
7.  ...}
8.}
9.main()
10. {static int var3;
11.  extern int v2;
12.  int v1, y;
13.  for(y=0; y<10; y++)
14.    { char var1='C';
15.      a=&v1;}
16. }
ARCHIVO2.C
17.static int aux;
18.int v2;
19.static int fun2( )
20.{ extern int v1;
21.  aux=aux+1;
22.   …
23. }
24.int fun3( )
25. { int aux;
26. aux=aux+1;
27. …
28. }
```

Aclaración:
Ident.= Identificador
T.V. = Tiempo de Vida
r-valor debe ser tomado al momento de la alocación en memoria
El alcance de los identificadores debe indicarse desde la línea siguiente a su declaración.

Identificador	**Tipo (L-value)**	**r-valor (alocación)**	**Alcance**	**T.V.**
v1(linea 1)	automática	0	2-3;9-11;20-28	1-28